Q5 – Race Conditions in a Cinema Seat-Booking System

Complete the Code – TODO 1

```
public class SeatBooking {
    private final boolean[] booked;
    private int bookingsMade = 0;
    SeatBooking(int seats) { booked = new boolean[seats]; }

    // TODO 1 (completed)
    synchronized boolean book(int seat) {
        if (!booked[seat]) {
            booked[seat] = true;
            bookingsMade++;
            return true;
        }
        return false;
    }

    public static void main(String[] args) throws InterruptedException
{
        SeatBooking screen = new SeatBooking(10);
        Thread[] users = new Thread[100];
        for (int i = 0; i < users.length; i++) {
            final int seat = i % 10;
            users[i] = new Thread(() -> screen.book(seat));
            users[i].start();
        }
        for (Thread t : users) t.join();
        System.out.println("Seats sold: " + screen.bookingsMade);
    }
}
```

Expected output: `Seats sold: 10`

Without `synchronized`: run it 10 times and record the results (e.g. sometimes > 10; `<fill>`). To make the race reliable, add `Thread.yield()` or `Thread.sleep(1)` between the check and the write.

Why it may sell more than 10: `book` is check-then-act: two threads can both read `booked[seat] == false` before either writes `true`. Both then book and increment. Also `bookingsMade++` is not atomic (read, add, write), so updates can be lost (the count can even be wrong in the other direction).

Student A – Race explained

Interleaving for seat 3: | Step | Thread 1 | Thread 2 | |---|---|---| | 1 | reads `booked[3]` = false | | | 2 | | reads `booked[3]` = false | | 3 | sets `booked[3]=true`, count++ | | | 4 | | sets `booked[3]=true`, count++ → **double booking** |

Critical section: the `if (!booked[seat]) { booked[seat] = true; bookingsMade++; }` block, the check, the write and the counter update, which must execute atomically.

Student B – Strategies

```
// Synchronized method
synchronized boolean book(int seat) { ... }

// Synchronized block (only the critical section)
boolean book(int seat) {
    // non-critical work (validation, logging) outside the lock
    synchronized (this) {
        if (!booked[seat]) { booked[seat] = true; bookingsMade++;
return true; }
    }
    return false;
}
```

- **Measuring:** time 10,000 bookings with `System.nanoTime()` for (a) unsynchronized, (b) method, (c) block; average over several runs and tabulate `<fill>`. For tiny critical sections the difference is small. The block version wins only if there's real non-critical work outside the lock.
- **Whole method vs critical section:** a smaller locked region means less contention and better throughput; the method lock is simpler and less error-prone.
- **Static synchronization:** `static synchronized` locks on the `Class` object, shared across all instances. It would be needed here if the seat data were `static` (shared by all screens). With per-screen instance data, it isn't needed and would needlessly serialize all 20 screens.
- **Intrinsic locks:** synchronized method → lock on `this`; synchronized block → lock on the object named in the parentheses (here `this`); static synchronized → lock on `SeatBooking.class`.

Joint decision (20 screens)

Use a separate `SeatBooking` object per screen with instance-level `synchronized` (or a per-seat lock/`AtomicBoolean[]` if contention on one screen is high). Different screens then book in parallel while each screen's seats stay consistent. A single global (static) lock would serialize all 20 screens for no benefit.

Alternative

```
// AtomicBoolean[] with CAS
AtomicBoolean[] seats; AtomicInteger sold = new AtomicInteger();
boolean book(int s) { if (seats[s].compareAndSet(false, true))
{ sold.incrementAndGet(); return true; } return false; }

// ConcurrentHashMap
ConcurrentHashMap<Integer,Boolean> m; boolean book(int s) { return
m.putIfAbsent(s, true) == null; }
```

```
// ReentrantLock
lock.lock(); try { ...critical section... } finally { lock.unlock(); }
```

- **Time per booking under contention:** `synchronized` / `ReentrantLock`: threads block and context-switch, so cost grows with the number of waiting threads; O(1) uncontended, but waiting time grows with contenders. CAS (`AtomicBoolean`): non-blocking, O(1) per attempt; with one winner per seat, failed attempts return immediately, so it scales better with per-seat granularity. `ConcurrentHashMap.putIfAbsent`: O(1) average with fine-grained internal locking/CAS.
- **Extra space for s seats:** `synchronized`: O(1) beyond the `boolean[s]` (the monitor lives in the object header). `AtomicBoolean[]`: s objects, O(s) with more overhead per seat than a primitive. `ReentrantLock`: O(1) (one lock), or O(s) for per-seat locks. `ConcurrentHashMap`: O(s) for entries and table overhead.